

Adaptive Generate-Rank-Verify: Inference-Time Search with Costly Verification

Mahdi Haghifam

Shaddin Dughmi
(University of Southern California)



Yusuf Hakan Kalaycı
(University of Southern California)

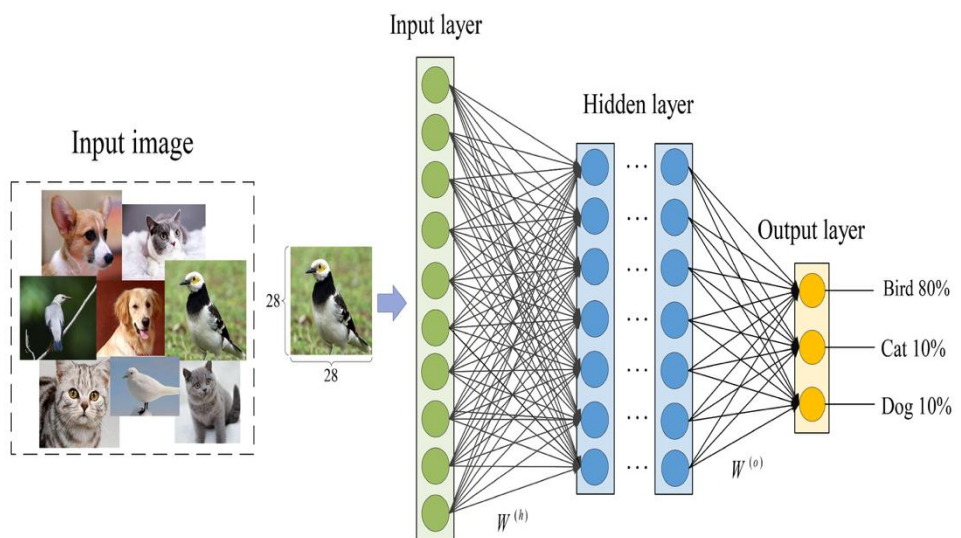


To appear at NeurIPS'26

Inference-Time Algorithm in LLMs

Google DeepMind

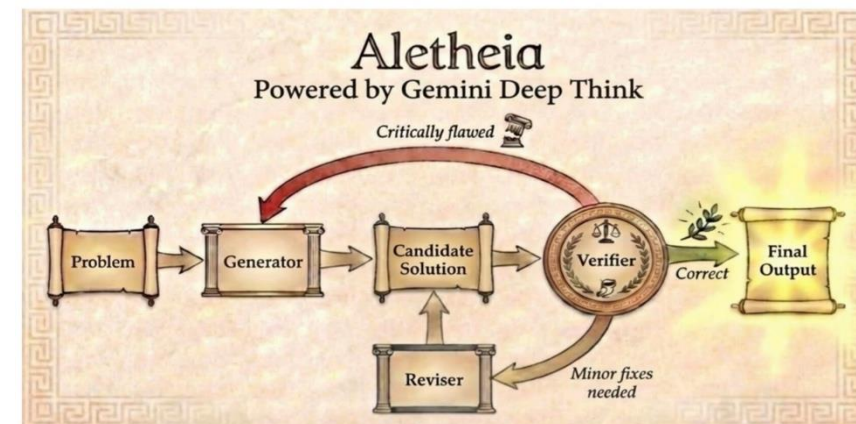
2026-2-11



Towards Autonomous Mathematics Research

Tony Feng^{*†}, Trieu H. Trinh, Garrett Bingham, Dawsen Hwang, Yuri Chervonyi, Junehyuk Jung[†], Joonkyung Lee[†], Carlo Pagano[†], Sang-hyun Kim[†], Federico Pasqualotto[†], Sergei Gukov[†], Jonathan N. Lee, Junsu Kim, Kaiying Hou, Golnaz Ghiasi, Yi Tay, YaGuang Li, Chenkai Kuang, Yuan Liu, Hanzhao (Maggie) Lin, Evan Zheran Liu, Nigamaa Nayakanti, Xiaomeng Yang, Heng-tze Cheng, Demis Hassabis, Koray Kavukcuoglu, Quoc V. Le^{*}, Thang Luong^{*}

^{*}Project leads, [†]Mathematicians. Work conducted under Google DeepMind.

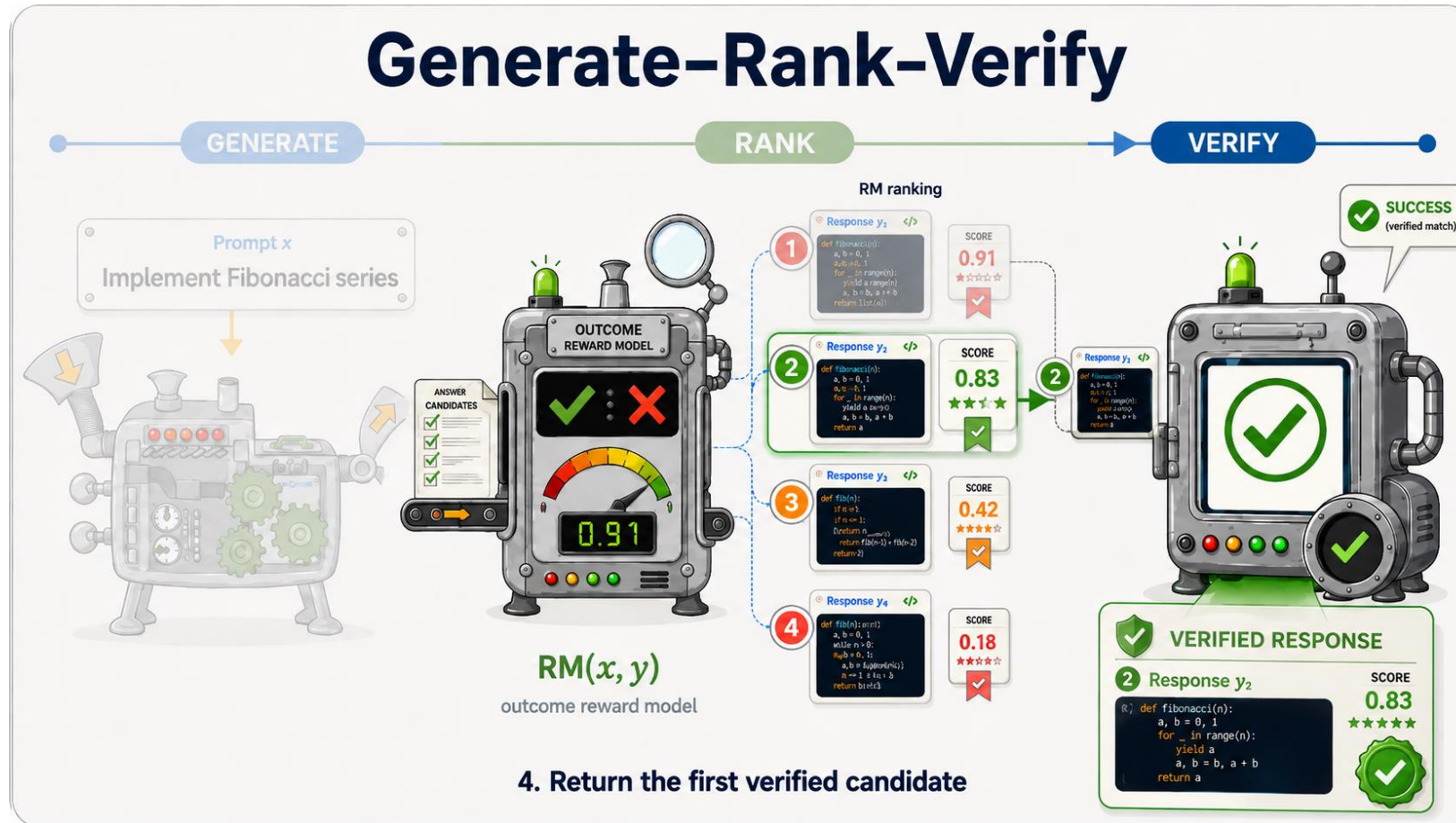


Using **one or more pre-trained models** as a **subroutine** to solve an instance better during inference-time (or test-time)

The landscape of the algorithm design is vast

best-of-N [Stiennon et al. '20, Nakano et al. '22], chain of thought [Wei et al. '22], self-consistency [Wang et al. '23], Tree of Thoughts [Yao et al. '23], outcome verifiers [Cobbe et al. '21], process reward models [Lightman et al. '24, Wang et al. '24], generative verifiers [Zhang et al. '25], execution-based selection [Li et al. '22, Chen et al. '23], test-time compute scaling [Snell et al. '25], ...

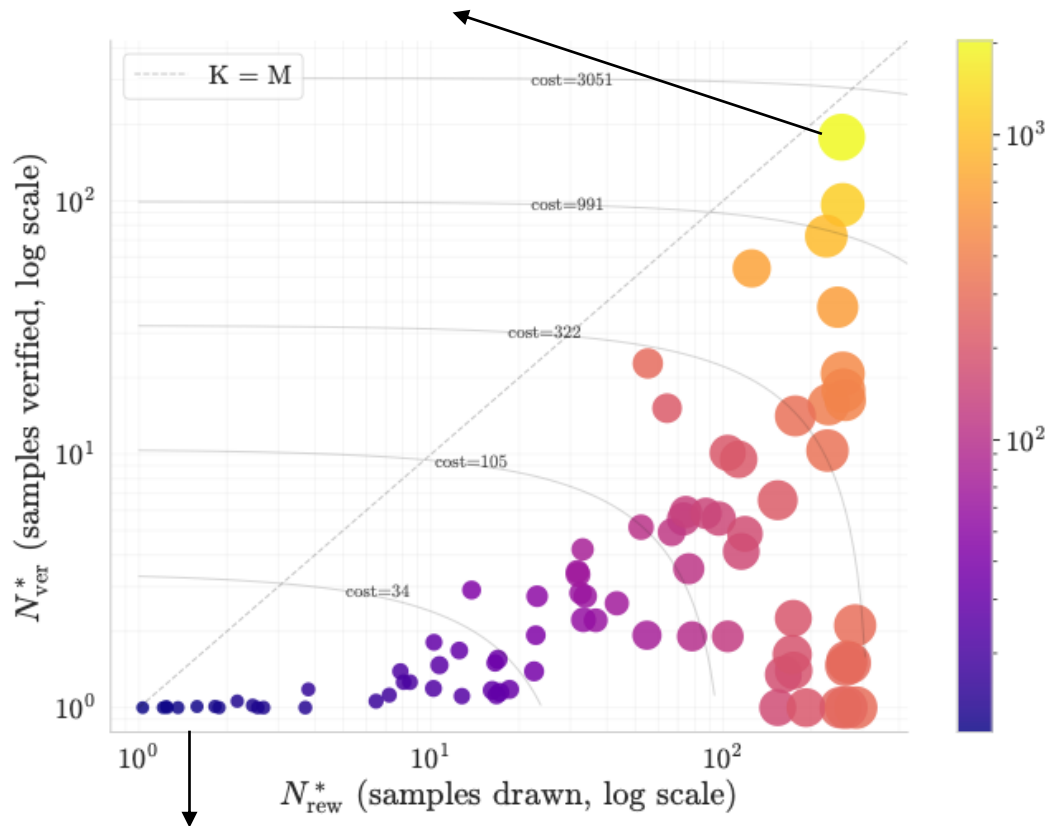
Generate-Rank-Verify Paradigm



Generate **N** samples, Rank by reward model, and **Verify** top k

Adaptivity in Inference-Time Search

Generate 500 responses
and verify almost all of them



Fixed budgets fail both ways
Overspend on the easy prompts, or run out on the hard ones.

Predicting difficulty offline is not enough
It assumes a stable prompt distribution.
[Damani et al.'24, Snell et al.'25]

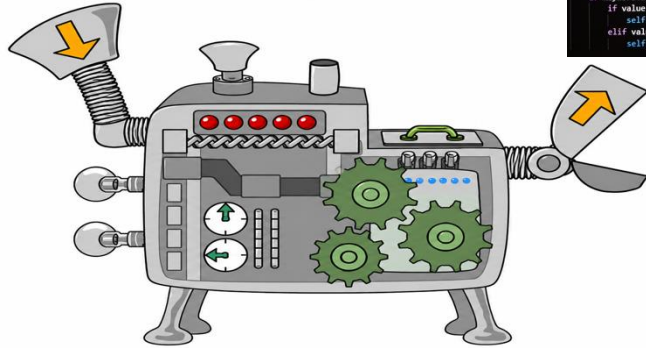
Generate 5 responses
and verify the top

Meta Question of This Talk:
Can we decide the **budget online**, **per prompt**, from an **uncalibrated reward model** and still **pay near-oracle cost**?

Problem Formalization: Active Search

Prompt x

Implement Fibonacci series



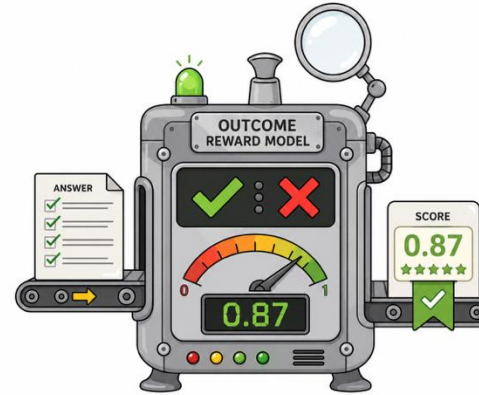
$$\pi(\cdot | x): \mathcal{X} \rightarrow \Delta(\mathcal{Y})$$

Language model or generator

Response y

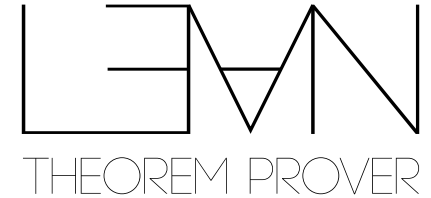
```
def send_header(self, keyword, value):
    """Send a MIME header to the headers buffer."""
    if self.request_version != 'HTTP/0.9':
        if not hasattr(self, 'headers_buffer'):
            self.headers_buffer = []
        self.headers_buffer.append(
            ("%s: %s\n" % (keyword, value)).encode
        )

    if keyword.lower() == 'connection':
        if value.lower() == 'close':
            self.close_connection = True
        elif value.lower() == 'keep-alive':
            self.close_connection = False
```



$$\text{RM}: \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$$

Learned outcome reward model



COMPILER

$$\text{Ver}: \mathcal{X} \times \mathcal{Y} \rightarrow \{0,1\}$$

Verifier

(proof checker, hidden-test execution)

Active Search Instance ($h, \mathcal{D}$)

- Fix a prompt $x \in \mathcal{X}$
- For a sampled response $Y \sim \pi(\cdot | x)$, define random variables $R = \text{RM}(x, Y)$ and $V = \text{Ver}(x, Y)$
- Define the **distribution of R** as $\mathcal{D}$
- Define **reward-verifier relationship** as $h(r) = \Pr(V = 1 | R = r)$ and $V = \text{Bern}(h(R))$

The verifier is deterministic; the randomness is in the LLM's sample, and $h(r)$ is the pass rate among samples scoring r .

If the reward model assigns assigns score r to a response, what is the prob of the verifier accepts it?

Problem Formalization: Active Search

prompt $x \in \mathcal{X}$
 $Y \sim \pi(\cdot | x)$, $R = \text{RM}(x, Y)$ and $V = \text{Ver}(x, Y)$
 $R \sim \mathcal{D}$
reward-verifier relationship $h(r) = \Pr(V = 1 | R = r)$

Definition: Active Search Algorithm $\mathcal{A}$

Given a prompt x . Repeat until a verifier returns 1, choosing one of two actions based on $\mathcal{A}$:

- 1- **Generate**: Draw $Y \sim \pi(\cdot | x)$, score $R = \text{RM}(x, Y)$, and add (Y, R) to the unverified pool. Pay cost c_{rew} .
- 2- **Verify**: Pick any (Y, R) from the unverified pool, query V . Pay cost c_{ver} .

If $V = 1$, the policy **stops** and **outputs** Y .

The cost of $\mathcal{A}$ is given by

$$J(\mathcal{A}; \mathcal{D}, h) = N_{\text{rew}} c_{\text{rew}} + N_{\text{ver}} c_{\text{ver}}$$

#generation + reward model query

#verifier call

Problem Formalization: Adaptive Active Search

What counts as doing well?

Distribution-Aware Benchmark: unavoidable cost

$J^*(\mathcal{D}, h)$: the expected cost of the algorithm that knows both $(\mathcal{D}, h)$

prompt $x \in \mathcal{X}$
 $Y \sim \pi(\cdot | x), R = \text{RM}(x, Y)$ and $V = \text{Ver}(x, Y)$
 $R \sim \mathcal{D}$
reward-verifier relationship $h(r) = \Pr(V = 1 | R = r)$

Can we assume nothing about h (reward-verifier relationship)?

$h \in \mathcal{H}$ (probabilistic concept class) the class is known to the algorithm, h is not.

Main Technical Question

Fix a class $\mathcal{H}$. Does there exist an active search algorithm $\mathcal{A}$, that knows $\mathcal{H}$ but not h , and a constant C such that for every $h \in \mathcal{H}$ and every $\mathcal{D}$,

$$\mathbb{E}[J(\mathcal{A}; \mathcal{D}, h)] \leq C \cdot J^*(\mathcal{D}, h)$$

Easy prompts $\rightarrow$ small $J^*(\mathcal{D}, h) \rightarrow$ cost of $\mathcal{A}$ also small

If $\mathcal{A}$ incurs a high cost for search $\rightarrow$ the distribution-aware benchmark also incurs a high cost

Outline of the Talk

- Adaptive Inference Time Algorithm



One fixed budget fits no one: easy prompts need 5 samples, hard ones need 500.

- ADAP algorithm
- Learning Theoretic Foundations
- Takeaways and Future Work

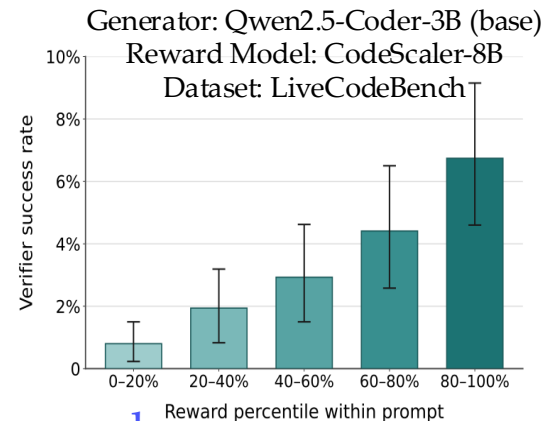
Our Proposed Algorithm: ADAP

Assumption: $h(r) = \Pr(V = 1 | R = r)$ is non-decreasing in r ($\mathcal{H}_{non-dec}$)
(our results can be extended to $(1 - \eta)g(r) < h(r) < (1 + \eta)g(r)$)

prompt $x \in \mathcal{X}$
 $Y \sim \pi(\cdot | x)$, $R = \text{RM}(x, Y)$ and $V = \text{Ver}(x, Y)$
 $R \sim \mathcal{D}$
reward-verifier relationship $h(r) = \Pr(V = 1 | R = r)$

What this buys: Verifying the top-ranked candidates is always the best use of the budget.

Simple strategy: verify only candidates whose reward is at least t .



$q_t = \Pr(R \geq t)$ how often we pay to verify $s_t = \Pr(R \geq t, V = 1)$ how often we succeed

Each round costs c_{rew} always, plus c_{ver} with probability q_t , and succeeds with probability s_t : $J_t = \frac{c_{rew} + c_{ver} q_t}{s_t}$

The benchmark knows the best threshold; ADAP does not.

Main Observation

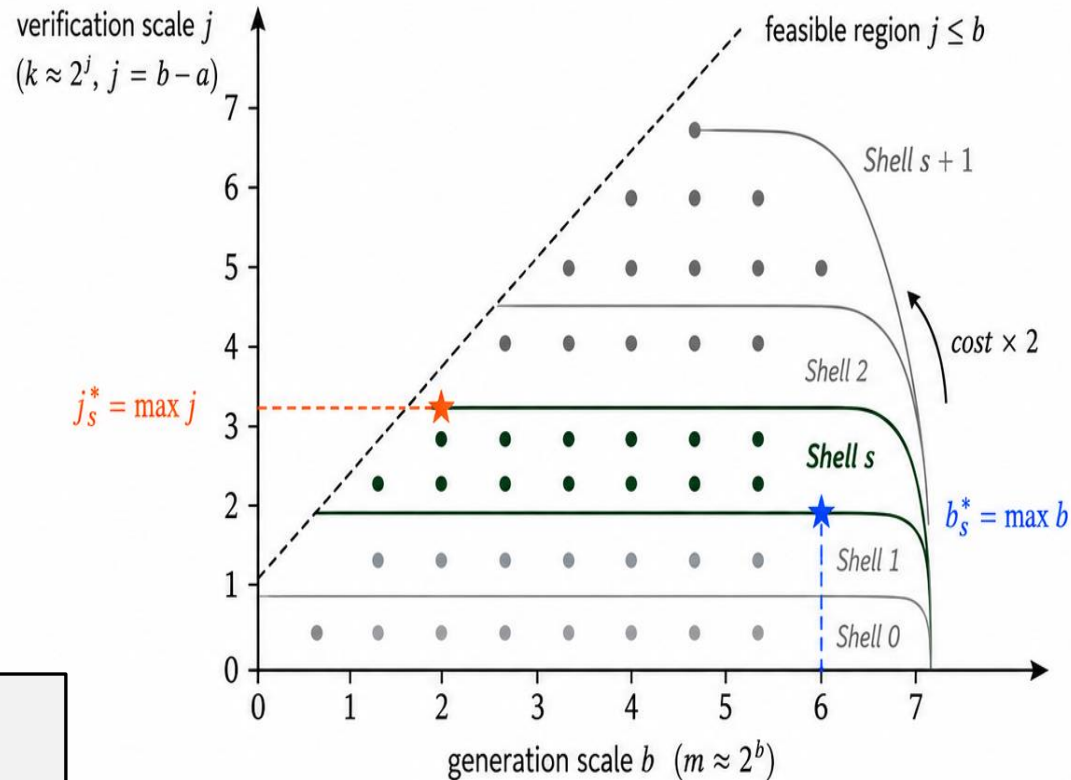
Suppose I tell you a good threshold t . Then generating on the order of $1/s_t$ responses and verifying on the order of q_t/s_t top-ranked responses finds a verified response with a constant probability.

$q_t \approx 2^{-a}$ and $s_t \approx 2^{-b}$ then the threshold corresponds to the generation budget $m \approx 2^a$ and $k \approx 2^{b-a}$.

Algorithm Overview

prompt $x \in \mathcal{X}$
 $Y \sim \pi(\cdot | x)$, $R = \text{RM}(x, Y)$ and $V = \text{Ver}(x, Y)$
 $R \sim \mathcal{D}$
 reward-verifier relationship $h(r) = \Pr(V = 1 | R = r)$

- 1 **Every dot is a guess (b, j)**
 generate $m \approx 2^b$, verify top $k \approx 2^j$
 cost: $c_{\text{rew}} 2^b + c_{\text{ver}} 2^j$
- 2 **Shells group guesses by cost**
 shell s : cost in $[2^s, 2^{s+1}) \cdot c_{\min}$
 each shell costs twice the previous
- 3 **One stage covers a whole shell**
 run its corner: **largest b** , **largest j**
 it dominates every guess in the shell



For shell S_s

- rightmost point $\rightarrow b_s^* \rightarrow$ generation budget
- topmost point $\rightarrow j_s^* \rightarrow$ verification budget

$$m_s = \lceil 2^{b_s^* + 1} \rceil$$

$$k_s = \lceil 6 \cdot 2^{j_s^*} \rceil$$

constants from the analysis

Doubling trick in 2D

before the oracle's shell: a geometric sum
 after it: rarely reached $\Rightarrow \mathbb{E}[\text{cost}] = O(J^*)$

Performance Guarantee of ADAP

prompt $x \in \mathcal{X}$
 $Y \sim \pi(\cdot | x)$, $R = \text{RM}(x, Y)$ and $V = \text{Ver}(x, Y)$
 $R \sim \mathcal{D}$
reward-verifier relationship $h(r) = \Pr(V = 1 | R = r)$

Theorem [Dughmi–Haghifam–Kalayci '25]

Assume $h \in \mathcal{H}_{\text{non-dec}}$. Then for every feasible prompt, every generator, and every induced $(h, \mathcal{D})$,

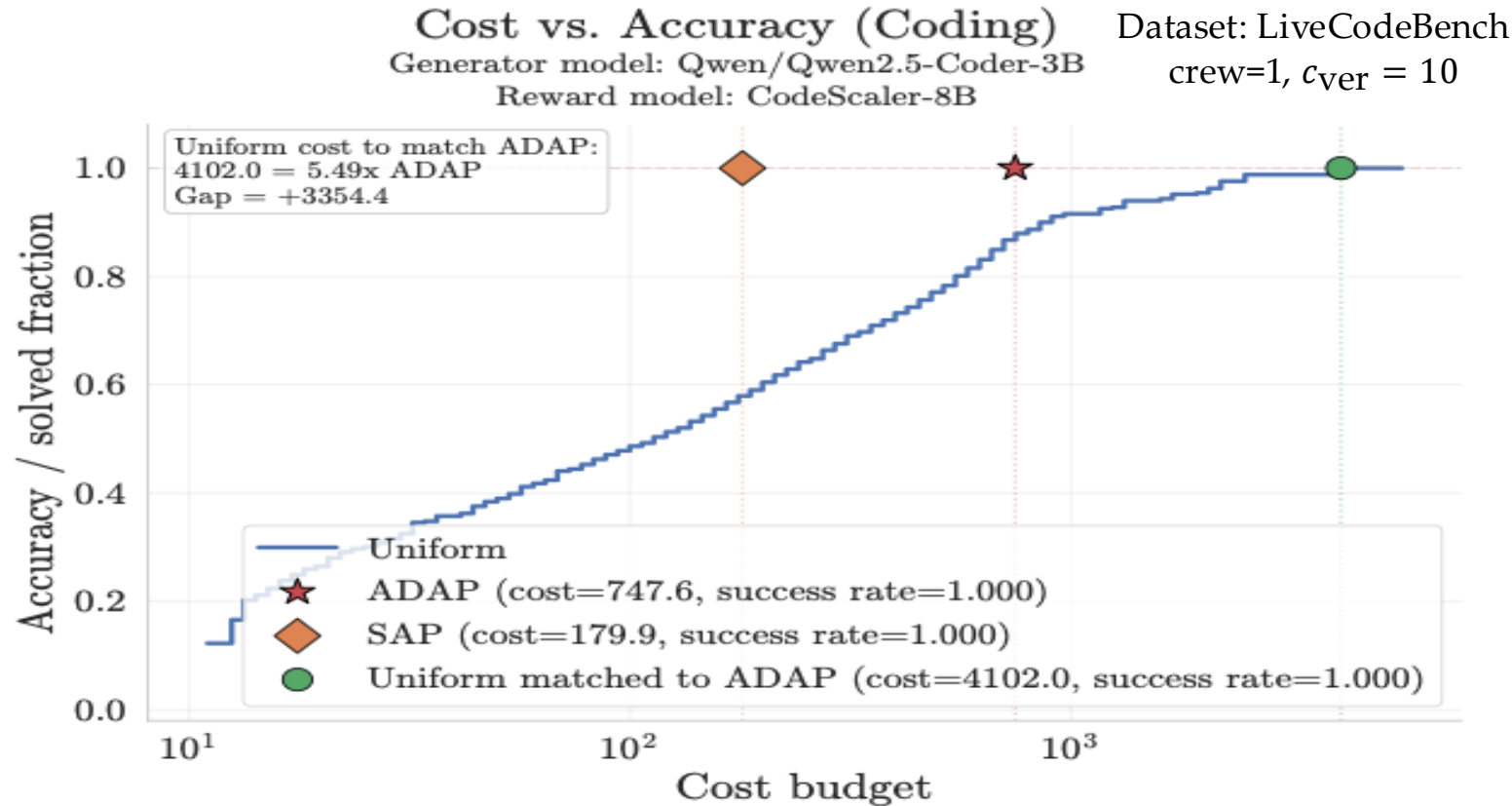
$$\mathbb{E}[J(\mathcal{A}_{\text{ADAP}}; h, \mathcal{D})] \leq 173 J^*(h, \mathcal{D})$$

ADAP knows neither h nor $\mathcal{D}$, and uses the reward model only to rank.

Easy prompts $\rightarrow$ small $J^*(\mathcal{D}, h) \rightarrow$ cost of $\mathcal{A}$ also small

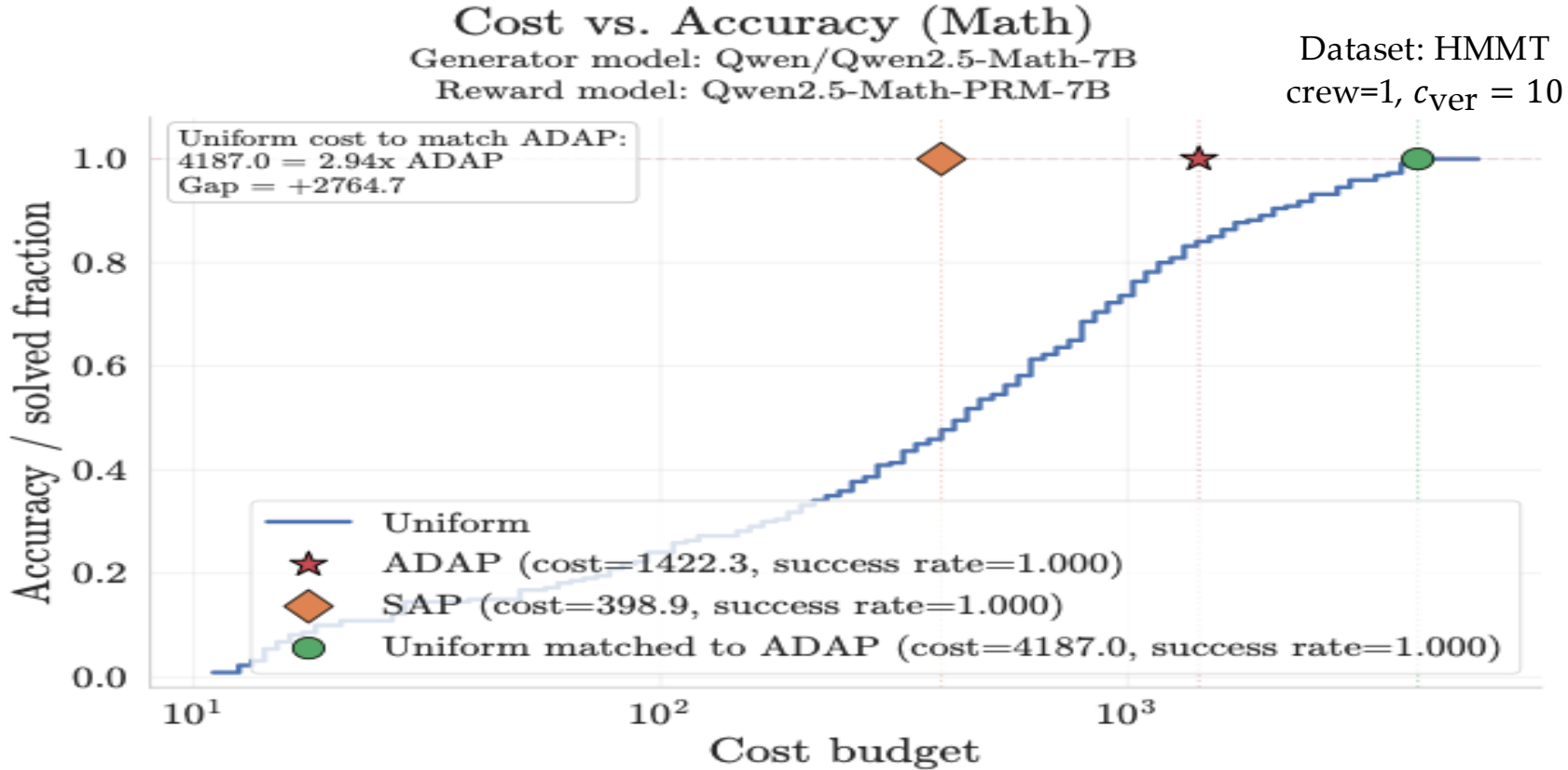
If $\mathcal{A}$ incurs a high cost for search $\rightarrow$ the distribution-aware benchmark also incurs a high cost

ADAP gets reliability at much lower cost - Coding



- **Uniform:** The blue curve represents the best possible fixed-budget strategy. For every cost budget, we search over all fixed pairs of generation and verification counts and report the pair that solves the most problems. So this is actually a strong baseline: it is the upper envelope of all fixed policies.
- **SampleAware (SAP):** For each prompt, we allocate the best (N_{rew}, N_{ver}) to minimize the budget. No realizable policy can reproduce it. It is a per-trial floor, not a competitor.

ADAP gets reliability at much lower cost - MATH



- **Uniform:** The blue curve represents the best possible fixed-budget strategy. For every cost budget, we search over all fixed pairs of generation and verification counts and report the pair that solves the most problems. So this is actually a strong baseline: it is the upper envelope of all fixed policies.
- **SampleAware (SAP):** For each prompt, we allocate the best $(N_{\text{rew}}, N_{\text{ver}})$ to minimize the budget. No realizable policy can reproduce it. It is a per-trial floor, not a competitor.

ADAP vs Clustering Benchmarks

Policy	Math (HMMT)					Coding (LiveCodeBench)				
	$\overline{N}_{\text{rew}}$	$\overline{N}_{\text{ver}}$	Avg. cost	Ratio	Succ.	$\overline{N}_{\text{rew}}$	$\overline{N}_{\text{ver}}$	Avg. cost	Ratio	Succ.
SAMPLEAWARE	103.7	29.5	398.9	$\times 0.28$	1.000	92.4	8.7	179.9	$\times 0.24$	1.000
ADAP	219.6	120.2	1422.3	$\times 1.00$	1.000	140.4	60.5	745.3	$\times 1.00$	1.000
UNIFORM C_{ADAP}	182.0	124.0	1422.0	$\times 1.00$	0.841	414.0	33.0	744.0	$\times 1.00$	0.878
DAP ₁	497.0	369.0	4187.0	$\times 2.94$	1.000	512.0	359.0	4102.0	$\times 5.50$	1.000
DAP ₂	—	—	2164.8	$\times 1.52$	1.000	—	—	2162.1	$\times 2.90$	1.000
DAP ₃	—	—	1792.7	$\times 1.26$	1.000	—	—	1582.1	$\times 2.12$	1.000
DAP ₅	—	—	1477.3	$\times 1.04$	1.000	—	—	1194.3	$\times 1.60$	1.000
DAP ₁₀	—	—	1169.4	$\times 0.82$	1.000	—	—	786.9	$\times 1.06$	1.000

- **DAP_k**: An oracle difficulty-adaptive policy. It sorts problems by their true pass rate, splits them into k difficulty classes, and gives each class its own fixed budget.
Not realizable: it needs per-problem difficulty up front. Larger k means finer stratification and lower cost.
- **Takeaway**: Given the very same budget (1422 on math, 745 on coding), the best fixed policy solves only 84% / 88% of problems — ADAP solves 100%, matching the 10-class oracle on coding ($\times 1.06$) with no difficulty labels.

Outline of the Talk

- Adaptive Inference Time Algorithm



One fixed budget fits no one: easy prompts need 5 samples, hard ones need 500.

- ADAP algorithm



Rank, then verify the top: ADAP stays within a constant factor of the oracle.

- Learning Theoretic Foundations
- Takeaways and Future Work

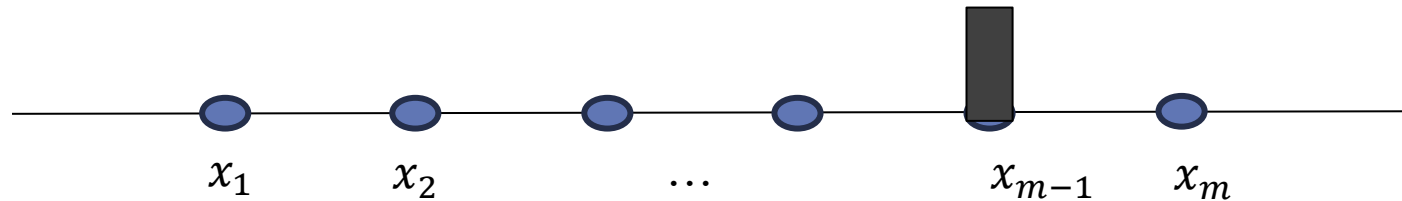
How much structure does active search need?

The proposed algorithm requires $h(r) = \Pr(V = 1 | R = r)$ be **approximately non-decreasing**.

Can we compete with a distribution-aware benchmark without any structural assumption?

Example [inspired by Balcan-Hanneke-Vaughan'10 & Hanneke-Yang'15]

Let the distribution $\mathcal{D}$ be uniform over $\{x_1, \dots, x_m\}$ and $\mathcal{H} = \{h_i(x_j) = 1[i = j]\}$



Cost of the distribution-aware benchmark

$$J^*(\mathcal{D}, h) = O(mc_{rew} + c_{ver})$$

Wait until you get x with $h(x) = 1$

Every active search algorithm

$$J(\mathcal{A}; \mathcal{D}, h) \geq \Omega(mc_{ver})$$

Needs to verify every point

For every active search algorithm

$$\frac{J(\mathcal{A}; \mathcal{D}, h)}{J^*(\mathcal{D}, h)} = \Omega(c_{ver}/c_{rew})$$

Learning-Theoretic View of Active Search

Definition: Centered Star Number

Let $\mathcal{Z}$ be an arbitrary input space. Let $\mathcal{H} \subseteq \{0,1\}^{\mathcal{Z}}$ be a binary concept class over $\mathcal{Z}$. Then, define

$$s_0(\mathcal{H}) = \max \{m \in \mathbb{N} : \exists z_1, \dots, z_m \in \mathcal{Z}, \exists h_1, \dots, h_m \in \mathcal{H} \text{ s.t. } h_i(z_j) = 1[i = j] \}.$$

Theorem [Dughmi-Haghifam-Kalayci'25]

Fix an arbitrary concept class with centered star number $s_0(\mathcal{H})$. Then, we have

$$\min_{\text{active search alg. } \mathcal{A}} \max_{(\mathcal{D}, h) \in \Delta(\mathcal{Z}) \times \mathcal{H}} \frac{J(\mathcal{A}; \mathcal{D}, h)}{J^*(\mathcal{D}, h)} = \Theta \left(\min \left\{ s_0(\mathcal{H}), \frac{c_{\text{ver}}}{c_{\text{rew}}} \right\} \right)$$

Requires using the structure of $\mathcal{H}$

Verify everything sequentially

Few Remarks:

1- Active search is easier than active learning even per instance given the mass of label 1 is not too small.

[Balcan-Hanneke-Vaughan'10 & Hanneke-Yang'15 & Balcan-Beygelzimer-Langford'06]

2- Does this characterization inspire how to select or train reward models for math/coding tasks?

3- 1-centered star number appears in [Hanneke'24].

Learning-Theoretic View of Active Search

Task

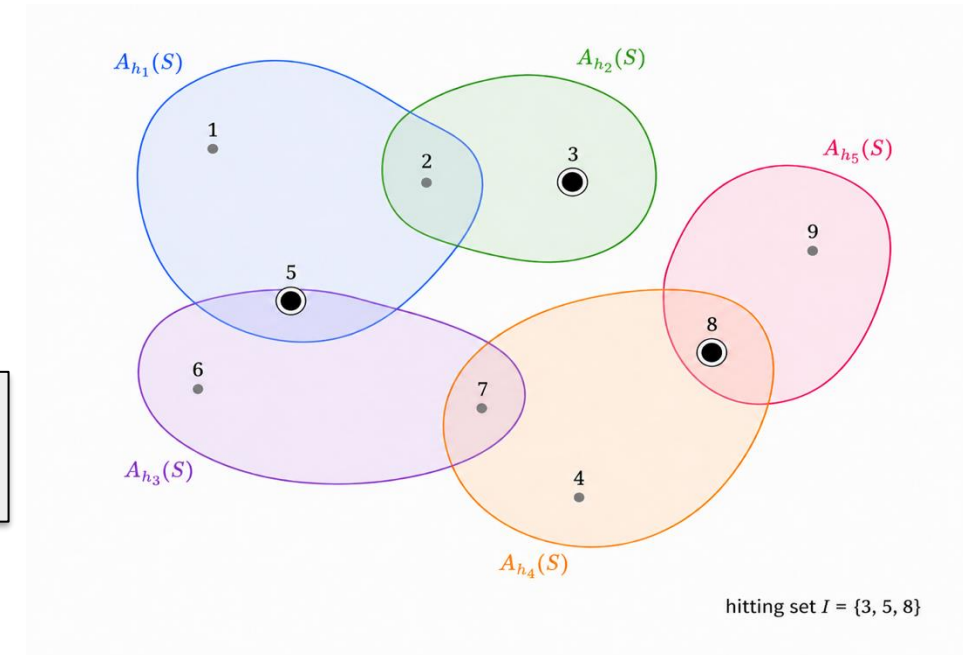
Given a large set of unlabeled samples $S = (Z_1, \dots, Z_n)$ with the minimum number of verification either

- 1) Find a Z_i such that $h^*(Z_i) = 1$
- 2) Discard the whole set if no label one exists

Step 1) For each $h \in \mathcal{H}$, form $A_h(S) = \{i \in [n] : h(Z_i) = 1\}$

Step 2) Find the minimum hitting set of $\cup_{h \in \mathcal{H}} A_h(S)$

Lemma [Dughmi-Haghifam-Kalayci'25]
The size of the hitting set is at most $s_0(\mathcal{H})$



Proof:

If Z_i is in the hitting set if there exists $h \in \mathcal{H}$ s.t. $h(Z_i) = 1$ and $\forall j \neq i, h(Z_j) = 0$



Hitting set forms a set we care about in the definition of centered star number.

Outline of the Talk

- Adaptive Inference Time Algorithm 

One fixed budget fits no one: easy prompts need 5 samples, hard ones need 500.

- ADAP algorithm 

Rank, then verify the top: ADAP stays within a constant factor of the oracle.

- Learning Theoretic Foundations 

No structure, no shortcut: the centered star number sets the price of adaptivity.

- Takeaways and Future Work

Takeaways and Future Directions

- **The model: active search.** Generate–rank–verify becomes a search problem: cheap noisy scores, one expensive exact check, find a correct answer for as little as possible.
- **The algorithm: ADAP.** Adapts to each prompt as it goes. Ranks with the reward model and uses nothing else. Stays within a constant factor of an oracle that knows the instance.
- **The limit: centered star number.** You cannot assume nothing about the reward model. We give a complete characterization: the adaptivity gap is $\min\{s_0(\mathcal{H}), c_{ver}/c_{rew}\}$,



GitHub Repository



arXiv Link

Our Proposed Algorithm: ADAP

Algorithm 1 $\mathcal{A}_{\text{ADAP}}$: Shellwise Active Search

Input: costs $c_{\text{ver}}, c_{\text{rew}} > 0$ and prompt x

Output: A response y with observed label $\text{Ver}(x, y) = 1$

```
1:  $c_{\min} \leftarrow \min\{c_{\text{rew}}, c_{\text{ver}}\}$ 
2:  $\mathcal{P} \leftarrow \emptyset$   $\triangleright$  pool of generated but unverified pairs
3: for  $s = 0, 1, 2, \dots$  do
4:    $\mathcal{S}_s \leftarrow \{(a, b) \in \mathbb{Z}_{\geq 0}^2 : a \leq b, 2^s c_{\min} \leq c_{\text{rew}} 2^b + c_{\text{ver}} 2^{b-a} < 2^{s+1} c_{\min}\}$ 
5:   if  $\mathcal{S}_s = \emptyset$  then
6:     continue
7:    $b_s^* \leftarrow \max\{b : (a, b) \in \mathcal{S}_s\}$ ,  $j_s^* \leftarrow \max\{b - a : (a, b) \in \mathcal{S}_s\}$ 
8:    $m_s \leftarrow \lceil 2^{b_s^*+1} \rceil$ ,  $k_s \leftarrow \lceil 6 \cdot 2^{j_s^*} \rceil$ 
9:   Draw fresh responses  $y_1, \dots, y_{m_s} \stackrel{\text{i.i.d.}}{\sim} \pi(\cdot | x)$ 
10:  Compute reward scores  $\{R_i = \text{RM}(x, y_i)\}_{i \in [m_s]}$  and add  $\mathcal{P} \leftarrow \mathcal{P} \cup \{(y_i, R_i) : i \in [m_s]\}$ 
11:  Relabel the elements of  $\mathcal{P}$  as  $(\tilde{y}_1, \tilde{R}_1), \dots, (\tilde{y}_{|\mathcal{P}|}, \tilde{R}_{|\mathcal{P}|})$  so that  $\tilde{R}_1 \geq \tilde{R}_2 \geq \dots \geq \tilde{R}_{|\mathcal{P}|}$ .
12:  for  $j = 1, \dots, \min\{k_s, |\mathcal{P}|\}$  do
13:    Query  $\text{Ver}(x, \tilde{y}_{(j)})$ 
14:    Remove  $(\tilde{y}_j, \tilde{R}_j)$  from  $\mathcal{P}$ 
15:    if  $\text{Ver}(x, \tilde{y}_{(j)}) = 1$  then
16:      return  $y_{(j)}$ 
```
